you have for `std::thread`, with an additional `interrupt()` function:

```
class interruptible_thread
{
public:
    template<typename FunctionType>
    interruptible_thread(FunctionType f);
    void join();
    void detach();
    bool joinable() const;
    void interrupt();
};
```

Internally, you can use `std::thread` to manage the thread itself and use some custom data structure to handle the interruption. Now, what about from the point of view of the thread itself? At the most basic level you want to be able to say, "I can be interrupted here"—you want an *interruption point*. For this to be usable without having to pass down additional data, it needs to be a simple function that can be called without any parameters: `interruption_point()`. This implies that the interruption-specific data structure needs to be accessible through a `thread_local` variable that's set when the thread is started, so that when a thread calls your `interruption_point()` function, it checks the data structure for the currently executing thread. We'll look at the implementation of `interruption_point()` later.

This `thread_local` flag is the primary reason you can't just use plain `std::thread` to manage the thread; it needs to be allocated in a way that the `interruptible_thread` instance can access, as well as the newly started thread. You can do this by wrapping the supplied function before you pass it to `std::thread` to actually launch the thread in the constructor, as shown in the next listing.

Listing 9.9. Basic implementation of `interruptible_thread`